Automating PET Analysis in Urban Transit Video with Camera-Specific Cyclist and Pedestrian Detection Tuning

1st Shayan Jalalipour
Computer Science
Portland State University
Portland, Oregon
shayan2@pdx.edu

2nd Banafsheh Rekabdar
Computer Science
Portland State University
Portland, Oregon
rekabdar@pdx.edu

***Abstract*—Post-Encroachment Time (PET) is useful for identifying near-miss interactions in transit conflict zones, but measuring it from fixed street-camera video still requires substantial manual review. We present an open-source RT-DETR and DeepSORT pipeline for cyclist and pedestrian detection, tracking, and PET analysis in Portland bus-stop video. The system addresses two practical barriers to automated PET analysis: limited labeled cyclist/pedestrian data for this camera setting, and camera-specific concept drift caused by angled mounting, object scale changes, and local geometry. Our pipeline combines semi-automated data curation, multi-stage RT-DETR fine-tuning, full-frame/top-region/perspective-warp inference, containment-aware NMS, per-class tracking, and a two-phase configuration search that tunes inference and tracker parameters without retraining the detector. On a labeled Portland camera clip, Auto-Tuned configuration improves mAP from 0.585 to 0.767 and cyclist recall from 44.6% to 80.4% over the manual configuration while using the same checkpoint.[1]**

***Index Terms*—Post-Encroachment Time, Cyclist Detection, Pedestrian Tracking, RT-DETR, Urban Traffic Safety, Configuration Tuning**

I. Introduction

Post-Encroachment Time (PET) measures the time gap between one road user leaving a conflict area and another entering it. In urban transit settings, low PET values can reveal locations where pedestrians, cyclists, buses, and other vehicles interact with little temporal separation. This makes PET attractive for proactive safety analysis because cities can study near misses without waiting for crash records. The bottleneck is labor: analysts must inspect long video recordings and reason about trajectories frame by frame.

Computer vision can reduce this cost, but street-camera PET analysis is not a solved object detection benchmark. General datasets such as COCO [1], Cityscapes [2], and EuroCity Persons [3] are valuable, but they do not fully match fixed Portland bus-stop cameras, where overhead angled mounting makes distant cyclists small and visually different from nearby cyclists. This creates concept drift within a single frame and across camera locations. It also exposes a data availability problem: commercial transit safety systems and their datasets are not generally reproducible, while new public camera locations require local labels, warp points, thresholds, and tracker settings.

In this paper, we present a practical pipeline for automating PET analysis from City of Portland bus-stop video. The objective is not only to detect cyclists and pedestrians, but to produce stable tracked trajectories that can drive PET heatmaps and risk summaries. To accomplish this, we combine semi-automated data curation, RT-DETR fine-tuning, camera-aware inference, per-class DeepSORT tracking, and a PET analysis module that converts tracked actor movement into conflict-zone outputs.

This paper provides three main contributions. First, we develop a workflow for building a local cyclist/pedestrian dataset from sparse street-camera footage, reducing the amount of empty video that must be manually reviewed. Second, we build a config-driven detection and tracking pipeline that handles small distant actors through top-region and perspective-warp inference while preserving separate tracker behavior for cyclists and pedestrians. Third, we introduce an Auto-Tuned camera-specific configuration search that improves the final PET input trajectories without retraining the detector. The rest of the paper reviews related work, provides relevant background, describes the method, reports experiments, and discusses limitations.

II. Related Work

Real-time object detection has moved from single-stage convolutional detectors such as YOLO [4] to transformer-based detectors such as DETR [5] and RT-DETR [6]. RT-MDet [7] shows that careful detector design can produce strong speed/accuracy tradeoffs, while RT-DETR removes several hand-designed components and remains practical for video inference. These detectors are typically validated on broad benchmarks, but cyclist and pedestrian safety analysis from fixed transit cameras depends on local geometry, small objects, and deployment-specific thresholds. Public urban

[1]Code available at https://github.com/shayan223/Cyclist-detection-using-YOLO

datasets help with pretraining, but they do not remove the need for local camera adaptation.

Multi-object tracking is the second requirement for PET because each actor must be followed through time. SORT [8] introduced a simple Kalman-filter and assignment framework, DeepSORT [9] added appearance-based re-identification, and ByteTrack [10] showed the value of associating low-confidence detections rather than discarding them. We use DeepSORT because appearance cues help preserve cyclist identities when boxes shift under occlusion, perspective change, or brief detector misses. However, our experiments show that cyclists and pedestrians need different tracker parameters because they move at different speeds and have different box dynamics.

Small-object detection and perspective correction are also central in this setting. SAHI [11] demonstrates that slicing can recover small distant objects, and homography-based traffic-camera work shows that inverse perspective mapping can make uncalibrated camera views more useful for traffic analysis [12]. Our code supports SAHI-style tiled inference, but the reported Auto-Tuned result disables that pass; the final configuration instead relies on full-frame, top-region, and perspective-warp inference. PET has been used as a surrogate safety measure in video-based traffic analysis, including studies that model the relationship between PET and signal timing from video data [13]. This prior work motivates PET as a useful measure, while our work focuses on the automation layer needed to make PET analysis reproducible for fixed bus-stop camera footage.

III. Background

PET is defined over a conflict area shared by two road users. If one actor leaves the conflict area at time t_{exit} and another actor enters it at time t_{entry}, then PET is $t_{entry} - t_{exit}$ when the second actor arrives after the first has cleared the area. Lower PET values indicate smaller temporal separation between road users, which is why PET is often treated as a surrogate safety metric: it does not require an actual collision, but it can reveal interactions where the margin for error was small. In this project, the conflict area can be represented as a grid cell, a selected cell, or an interactively drawn polygon over the camera frame.

Automated PET analysis requires more than per-frame object detection. A detector can identify cyclists and pedestrians in individual frames, but PET depends on entry and exit times, which require stable actor identities across time. Tracking failures can therefore change PET measurements even when per-frame detection appears reasonable. For this reason, our pipeline treats detection, duplicate suppression, per-class tracking, and PET computation as one connected workflow rather than independent post-processing steps.

The other relevant background issue is camera-specific concept drift. In fixed transit cameras, object appearance can change dramatically inside a single frame because actors near the top of the image appear smaller, more compressed, and more affected by perspective than actors near the bottom. Between camera locations, mounting height, viewing angle, lighting, shelter placement, and sidewalk geometry also change. Homography-based perspective warp helps normalize part of this geometry by mapping a selected image quadrilateral to a rectified view, while per-class tracking allows cyclist and pedestrian motion to be handled differently after detection.

IV. Materials and Methods

A. Dataset and Data Preparation

The local Portland cyclist/pedestrian dataset is stored in YOLO format with two classes, `cyclist` and `pedestrian`. The current dataset contains 3,121 images: 2,339 train, 520 validation, and 262 test images. Frames are drawn from fixed City of Portland bus-stop cameras and split with the project splitter using a 70%/20%/10% train/validation/test ratio. To reduce the amount of empty footage that must be reviewed, `movement_window.py` uses MOG2 background subtraction, polygon ROI selection, temporal persistence filtering, and aspect-ratio, solidity, and compactness heuristics to pause on likely activity. Human labels are then saved in YOLO format with class toggling between cyclists and pedestrians.

To increase coverage before local fine-tuning, the project includes a dataset combiner that filters YOLO-format sources to cyclist/pedestrian classes, maps aliases such as `person` and `people` to `pedestrian`, maps `bike`, `bicycle`, and `biker` to `cyclist`, and removes duplicate images through MD5 hashes. A separate augmentation script can expand training data with flips, HSV jitter, translation, scale, perspective warp, large-scale jitter, top-region blur, and copy-paste of upper-frame objects, while validation and test splits are copied unchanged.

B. Detection, Tracking, and PET Pipeline

We fine-tune RT-DETR with the Ultralytics framework using the project training script defaults: 100 epochs, batch size 8, image size 640, initial learning rate 0.001, patience 10, mosaic 1.0, mixup 0.15, random erasing 0.4, horizontal flip 0.5, and RandAugment. The training curves in Fig. 1 show the detector learning the curated cyclist/pedestrian task: training and validation losses generally decrease, while precision, recall, and mAP improve over training. This is an important part of the story, because it indicates that the detector can fit the available labeled data. The remaining deployment challenge is not simply failed training, but the concept drift introduced by fixed street-camera geometry and camera-specific operating conditions. The trained checkpoint is fixed during all configuration-search experiments.

At inference time, the video frame is downscaled to 960×540 for the final Auto-Tuned configuration. The pipeline runs a full-frame detector pass, a top-region pass over the upper 54% of the frame, and a warp pass using four configured homography points mapped to a 960×480 rectified view. Detections from these passes are merged, clipped to frame bounds, filtered by class-specific confidence floors, and passed

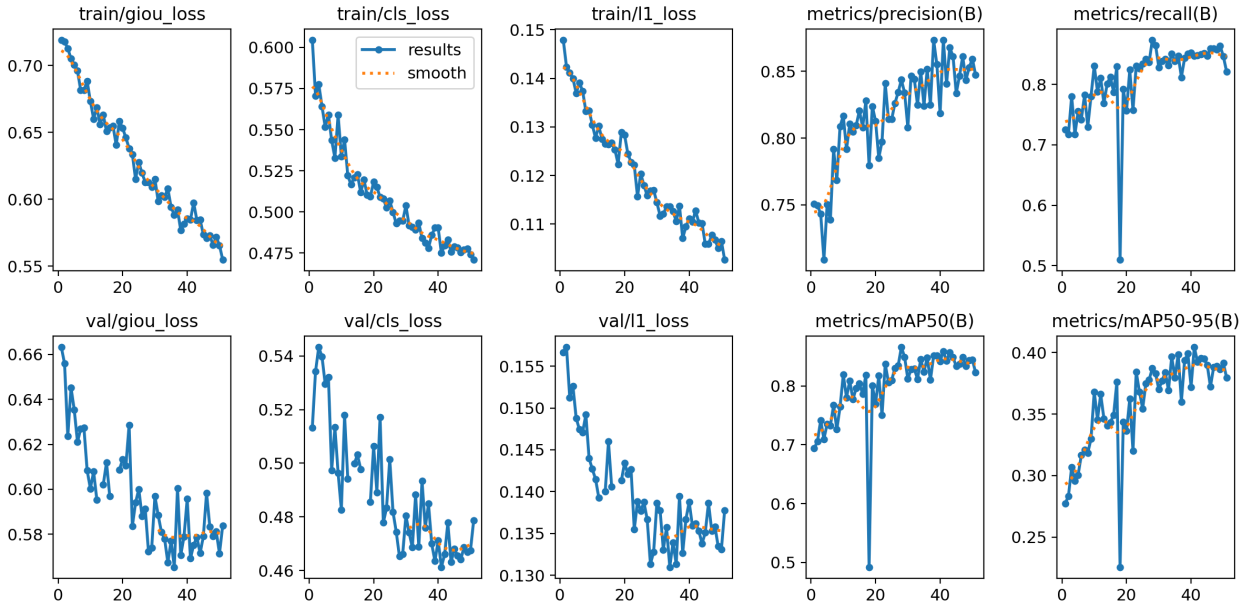

Fig. 1: RT-DETR training curves on the curated cyclist/pedestrian dataset. Losses decrease and validation precision, recall, and mAP improve, showing that the detector learns the labeled task. The later camera-specific tuning results show that strong training behavior still does not remove deployment-time concept drift from angled street-camera footage.

through hard NMS with containment cleanup followed by optional Gaussian Soft-NMS. The final Auto-Tuned NMS settings are hard IoU 0.21, containment fraction 0.57, Soft-NMS IoU 0.31, and $\sigma = 0.20$.

Tracking uses one DeepSORT instance per class. The Auto-Tuned cyclist tracker uses `max_age`=7, `n_init`=1, `max_iou_distance`=0.84, `nms_max_overlap`=0.15, and `max_cosine_distance`=0.80. The pedestrian tracker uses `max_age`=19, `n_init`=3, `max_iou_distance`=0.40, `nms_max_overlap`=0.15, and `max_cosine_distance`=0.57. This split reflects the observation that cyclists need quick confirmation and permissive association, while pedestrians benefit from tighter matching.

PET is computed from tracked actor histories. The pipeline uses the bottom-center of each tracked box as the ground-contact anchor and a small footprint rectangle, 40% of box width by 20% of box height, around that anchor. Conflict zones can be analyzed as a grid, a selected grid cell, or an interactively drawn polygon. For each pair of visits to a conflict region, PET is the time between the earlier actor's exit frame and the later actor's entry frame. Events are binned into 0–1.5 s, 1.5–3 s, 3–5 s, and 5+ s, then exported as CSV, heatmaps, and risk-over-time plots.

C. Auto-Tuned Configuration Search

Each camera has different lighting, scale, mounting, and conflict geometry, so a single YAML configuration does not transfer reliably. We therefore tune only inference and tracking parameters while keeping the detector checkpoint fixed. Algorithm 1 summarizes the search.

Algorithm 1 Camera-Specific Auto-Tuning

Input: seed YAML, detector checkpoint, labeled clip, $N = 50$, $M = 30$
Sample N broad random YAMLs over confidence, IoU, top-region ratio, NMS, class thresholds, and tracker parameters
for each sampled YAML **do**
    Run inference on the labeled clip and evaluate at IoU 0.50
    Score trial by AP, with a penalty when mean matched IoU < 0.65
end for
Select top-K seeds from the leaderboard
Allocate M refinement trials across seeds, weighted by score
Draw Gaussian perturbations around each seed and evaluate them
Output: best camera-specific YAML and leaderboard

V. Experimental Results and Discussion

A. Experimental Setup and Metrics

We evaluate on the Camera 1 Trim 5 Portland clip using a ground-truth CSV with 544 labeled frame rows. Three configurations are compared with the same detector checkpoint: Bare bones detector-only inference, Manual Trim 5, and Auto-Tuned. Evaluation uses IoU threshold 0.50. We report mAP, class AP, final precision/recall, matched boxes, and matched-box IoU statistics. The Auto-Tuned run used seed 42, 50

exploration trials, and 30 refinement trials around the top search seeds.

Intersection over Union (IoU) measures the overlap between a predicted box and a ground-truth box divided by their union. A prediction is counted as matched when it has the correct class and IoU at least 0.50 with an unmatched ground-truth box. Precision measures how many predicted boxes are correct, while recall measures how many ground-truth boxes are recovered. Average precision (AP) summarizes the precision-recall curve for one class, and mAP averages AP across the evaluated classes. Matched boxes report the number of true-positive detections and are useful here because cyclist recall is one of the main bottlenecks for PET trajectory quality.

TABLE I: Detection performance on the labeled Portland camera clip at IoU=0.50.

Configuration	mAP	Cyclist AP	Cyclist Rec.	Matched
Bare bones	0.542	0.364	0.370	317
Manual Trim 5	0.585	0.455	0.446	356
Auto-Tuned	**0.767**	**0.818**	**0.804**	**481**

Table I shows that the manually configured multi-pass pipeline improves modestly over bare-bones inference, but the Auto-Tuned configuration produces the main gain. mAP increases from 0.585 to 0.767, a 31.1% relative improvement over the manual configuration, and cyclist recall rises from 44.6% to 80.4%. Because the detector checkpoint is unchanged, the improvement comes from camera-specific inference, NMS, and tracker configuration rather than additional model training. Fig. 2 adds the matched-box IoU distribution, which checks that the higher match count does not come only from weaker localization.

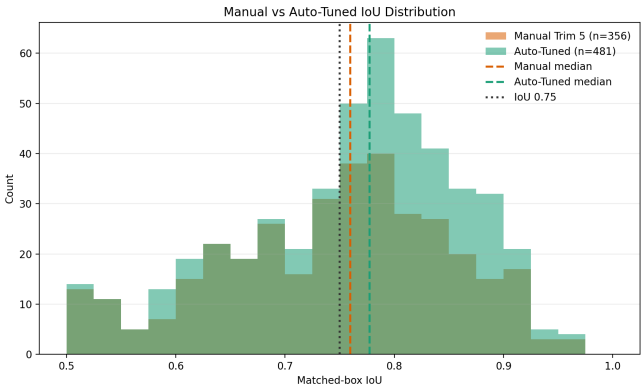

Fig. 2: Matched-box IoU distribution for Manual Trim 5 and Auto-Tuned detections. Auto-Tuned increases the matched-box count from 356 to 481 while shifting the median IoU above the manual median.

The tuning results reveal why camera-specific configuration matters even after successful training. The best configuration lowers the global confidence floor to 0.50 and the cyclist class floor to 0.57, allowing more distant cyclists to survive until tracker association and NMS. It also uses a high detector IoU setting, tight containment-aware hard NMS, and a permissive cyclist appearance threshold. These choices are not obvious by inspection because changing one threshold can either recover small cyclists or create duplicate tracks. The two-phase search provides a reproducible way to find a workable balance between a detector that has learned the class appearance and a camera view that still shifts the apparent scale, shape, and confidence of the same actor classes.

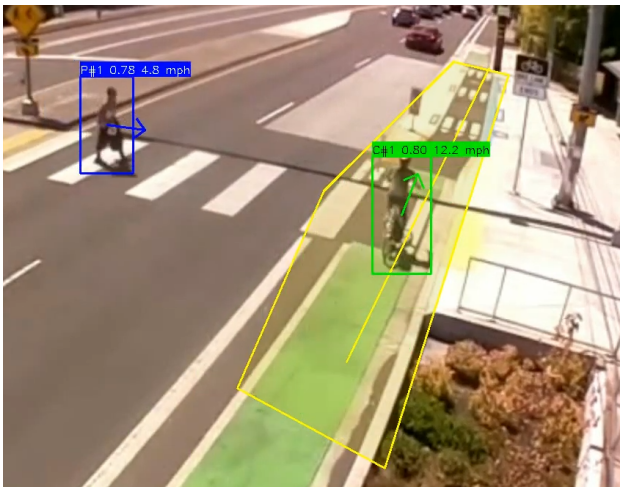

Fig. 3: Qualitative detection and tracking sample from the Portland bus-stop camera. The image shows detected pedestrian and cyclist actors, estimated direction, and the conflict area used for PET analysis.

The pipeline also produces PET artifacts for analyst review. PET heatmaps identify conflict-zone locations where low temporal separation clusters, as shown in Fig. 4, and risk plots summarize low-PET events over time.

Fig. 4: PET heatmap overlay for the bus-stop camera. The image is rotated to fit the column while preserving the camera-frame overlay used to identify where low temporal separation clusters.

Current limitations are important. The quantitative experiment is camera-specific, the same labeled clip drives tuning and evaluation, and PET quality depends on detector/tracker stability. Future work should evaluate on additional cameras, reserve separate tuning and test clips, add nighttime footage, and compare DeepSORT with ByteTrack-style association.

VI. Conclusion

We presented an open-source pipeline for automating PET analysis from fixed urban transit video. The system combines semi-automated data curation, RT-DETR fine-tuning, camera-aware inference, per-class DeepSORT tracking, PET heatmaps, and a two-phase Auto-Tuned configuration search. On a labeled Portland bus-stop clip, Auto-Tuned configuration improved mAP from 0.585 to 0.767 and cyclist recall from 44.6% to 80.4% without retraining the detector, showing that camera-specific configuration is a major lever for concept-drifted street-camera analysis. The next step is broader validation across more locations and conditions so PET automation can support scalable transit safety review.

References

[1] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755. [Online]. Available: https://arxiv.org/abs/1405.0312

[2] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The Cityscapes Dataset for Semantic Urban Scene Understanding," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 3213–3223. [Online]. Available: https://arxiv.org/abs/1604.01685

[3] M. Braun, S. Krebs, F. Flohr, and D. M. Gavrila, "The EuroCity Persons Dataset: A Novel Benchmark for Object Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 8, pp. 2772–2780, 2021. [Online]. Available: https://arxiv.org/abs/1805.07193

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. [Online]. Available: https://arxiv.org/abs/1506.02640

[5] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *European Conference on Computer Vision (ECCV)*, 2020, pp. 213–229. [Online]. Available: https://arxiv.org/abs/2005.12872

[6] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs Beat YOLOs on Real-time Object Detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16 965–16 974. [Online]. Available: https://arxiv.org/abs/2304.08069

[7] C. Lyu, W. Zhang, H. Huang, Y. Zhou, Y. Wang, Y. Liu, S. Zhang, and K. Chen, "RTMDet: An Empirical Study of Designing Real-Time Object Detectors," *arXiv preprint arXiv:2212.07784*, 2022. [Online]. Available: https://arxiv.org/abs/2212.07784

[8] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in *2016 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2016, pp. 3464–3468. [Online]. Available: https://arxiv.org/abs/1602.00763

[9] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in *2017 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2017, pp. 3645–3649. [Online]. Available: https://arxiv.org/abs/1703.07402

[10] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *European Conference on Computer Vision (ECCV)*, 2022, pp. 1–21. [Online]. Available: https://arxiv.org/abs/2110.06864

[11] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection," in *2022 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2022, pp. 966–970. [Online]. Available: https://arxiv.org/abs/2202.06934

[12] M. Zhu, S. Zhang, Y. Zhong, P. Lu, H. Peng, and J. Lenneman, "Monocular 3D Vehicle Detection Using Uncalibrated Traffic Cameras through Homography," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 13 469–13 475. [Online]. Available: https://arxiv.org/abs/2103.15293

[13] Z. Islam, M. Abdel-Aty, A. Goswamy, A. Abdelraouf, and O. Zheng, "Modelling the Relationship Between Post Encroachment Time and Signal Timings Using UAV Video Data," *Transportation Research Record*, vol. 2677, no. 5, pp. 567–582, 2022. [Online]. Available: https://arxiv.org/abs/2210.05044